Introdução à Programação e à Ciência de Dados

- Prof. Marcelo Rudek
- Prof. Helon Vicente Hultmann Ayala

CALENDÁRIO

Sala	Disciplina	Professor	Dia	Horário	Março				Abril					Maio	
Online	Introdução à Programação e à Ciência de Dados	Profs. Helon e Marcelo	Segunda	14:00-18:00	9	16	23	30	6	13	20	27		4	

EMENTA

Fundamentos da programação:

Manipulação de vetores, matrizes; leitura de dados; gráficos; formatação de figuras para publicações; plataformas para programação na nuvem (Google Colab).

Ciência de dados:

Fundamentos de estatística e probabilidade; análise exploratória de dados: Preparação de dados, Visualização, Transformação, Geração, Seleção de dados; Análise de correlação; Redução de dimensionalidade; Aplicações de ciência de dados.

AVALIAÇÃO:

Introd. Programação (50% - Entrega Exercícios) + Ciência de Dados (50%)

Python é uma linguagem de programação de alto nível, interpretada e de propósito geral, bastante utilizada atualmente por sua sintaxe simples e legível (intuitiva). Criada em 1991, Python permite clareza na escrita do código, permitindo o desenvolvimento rápido e eficiente de software. Ela suporta múltiplos paradigmas de programação, como orientação a objetos e imperativa. Possui uma vasta biblioteca de funções e uma comunidade ativa, tornando-a amplamente utilizada em diversas áreas como desenvolvimento web, ciência de dados, inteligência artificial, automação, entre outras.

O **MATLAB** é um ambiente de desenvolvimento para realizar cálculos numéricos, análise de dados, modelagem matemática e simulação. MATLAB é uma linguagem de alto nível voltada principalmente para computação científica e engenharia, com forte suporte para operações matriciais, visualização de dados e algoritmos avançados. Ele inclui um ambiente interativo e diversas bibliotecas especializadas em áreas como processamento de sinais, aprendizado de máquina e controle de sistemas. É amplamente utilizado em universidades, pesquisa e indústrias para resolver problemas complexos de forma eficiente e intuitiva

1. Operadores Aritméticos

Usados para realizar operações matemáticas:

Operador	Descrição	Exemplo (`a = 10`, `b = 3`)	Resultado
`+`	Adição	`a + b`	`13`
`-`	Subtração	`a - b`	`7`
`*`	Multiplicação	`a * b`	`30`
`/`	Divisão	`a / b`	`3.333...`
`//`	Divisão inteira	`a // b`	`3`
`%`	Módulo (resto da divisão)	`a % b`	`1`
`**`	Exponenciação	`a ** b`	`1000`

2. Operadores Condicionais (ou Relacionais)

Usados para comparar valores e retornam True ou False:

Operador	Descrição	Exemplo (`a = 10` , `b = 3`)	Resultado
`==`	Igual a	`a == b`	`False`
`!=`	Diferente de	`a != b`	`True`
`>`	Maior que	`a > b`	`True`
`<`	Menor que	`a < b`	`False`
`>=`	Maior ou igual a	`a >= b`	`True`
`<=`	Menor ou igual a	`a <= b`	`False`

3. Operadores Lógicos

Usados para combinar expressões condicionais:

Operador	Descrição	Exemplo (`x = True` , `y = False`)	Resultado
`and`	Retorna `True` se ambas as expressões forem verdadeiras	`x and y`	`False`
`or`	Retorna `True` se pelo menos uma expressão for verdadeira	`x or y`	`True`
`not`	Inverte o valor lógico da expressão	`not x`	`False`

1. Tabela Lógica do `AND` (E lógico)

O operador **AND** retorna `True` apenas se **ambas** as entradas forem `True`.

A	B	A and B
`False`	`False`	`False`
`False`	`True`	`False`
`True`	`False`	`False`
`True`	`True`	`True`

2. Tabela Lógica do `OR` (OU lógico)

O operador **OR** retorna `True` se **pelo menos uma** das entradas for `True`.

A	B	A or B
`False`	`False`	`False`
`False`	`True`	`True`
`True`	`False`	`True`
`True`	`True`	`True`

3. Tabela Lógica do `NOT` (Negação)

O operador **NOT** inverte o valor lógico da entrada.

A	not A
`False`	`True`
`True`	`False`

EXEMPLO: expressão booleana que usa os operadores **AND (E)**, **OR (OU)** e **NOT (NÃO)** em Python:

```
a = True
b = False
c = True

resultado = (a and b) or not c
print(resultado)
```

Explicação passo a passo:

1.(a and b): True and False → retorna False.
2.not c: not True → retorna False.
3.(False or False): False → Resultado final.

Tipos Numéricos

Usados para representar números:

- **int** → Números inteiros (ex: 10, -5, 1000)
- **float** → Números decimais/ponto flutuante (ex: 3.14, -0.5, 2.0)
- **complex** → Números complexos (ex: 2 + 3j, -1j)

Exemplo:

```
x = 10      # int
y = 3.14    # float
z = 2 + 3j  # complex
```

Tipo Booleano (bool)

Representa valores lógicos: True ou False.

Exemplo:

```
a = True
b = False
```

Tipo Texto (str)

Representa cadeias de caracteres (strings).

Exemplo:

```
nome = "Python"
frase = 'Olá, turma!'
```

Tipos de Sequência

Estruturas que armazenam múltiplos valores:

- **list** → Lista mutável (ex: [1, 2, 3])
- **tuple** → Tupla imutável (ex: (4, 5, 6))
- **range** → Sequência numérica (ex: range(1, 10))

Exemplo:

```
lista = [1, 2, 3, "Python"]  # list
tupla = (4, 5, 6)            # tuple
seq = range(1, 10, 2)        # range (1 até 10, incremento de 2 em 2)
```

Tipos de Conjuntos (set e frozenset)

Armazenam valores únicos:

- **set** → Conjunto mutável (ex: {1, 2, 3})
- **frozenset** → Conjunto imutável (ex: frozenset({4, 5, 6}))

Exemplo:

```
conjunto = {1, 2, 3, 4}
conjunto_imutavel = frozenset([5, 6, 7])
```

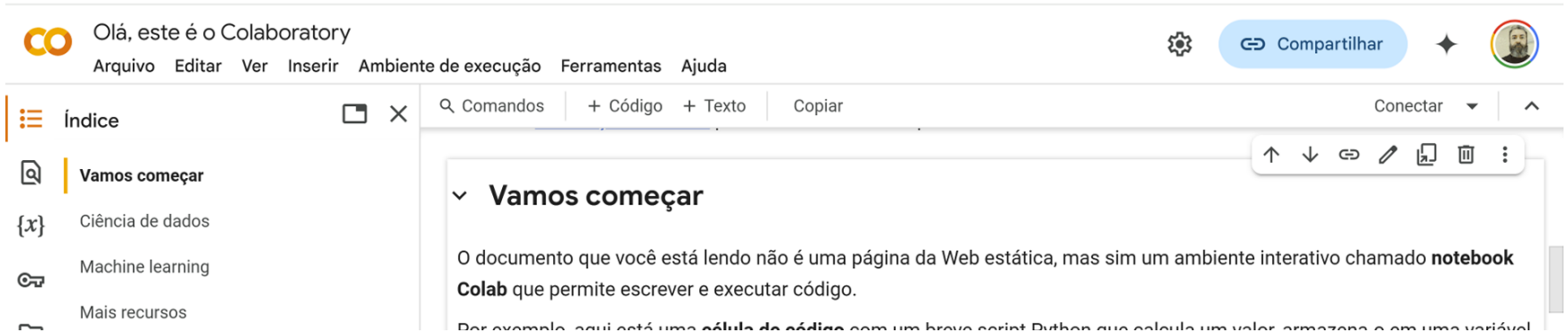
Tipo de Mapeamento (dict)

Estrutura de chave-valor.

Exemplo:

```
dicionario = {"nome": "Alice", "idade": 25}
```

Google Colab (*Google Colaboratory*)



https://colab.research.google.com/

O **Google Colab** (*Google Colaboratory*) é uma ferramenta gratuita baseada na nuvem que permite rodar código **Python** diretamente no navegador, sem precisar instalar um interpretador no computador.

Principais Características do Google Colab:

✅ **Execução na nuvem** → Não exige configuração local, pois usa servidores do Google.

✅ **Suporte a GPU e TPU** → Permite aceleração com placas gráficas para cálculos mais rápidos (útil para IA e Machine Learning).

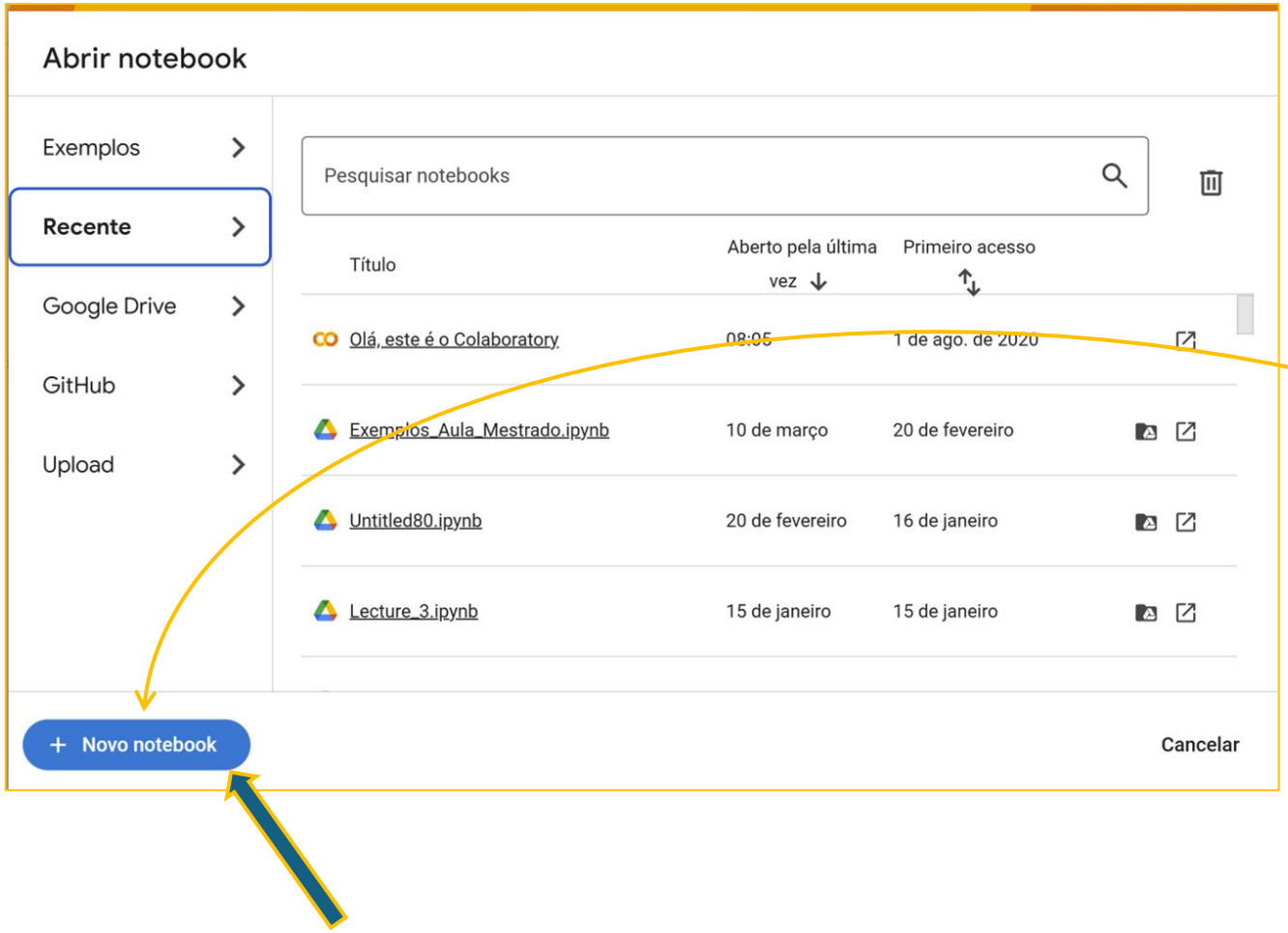
✅ **Integração com Google Drive** → Facilita salvar e carregar arquivos diretamente do Drive.

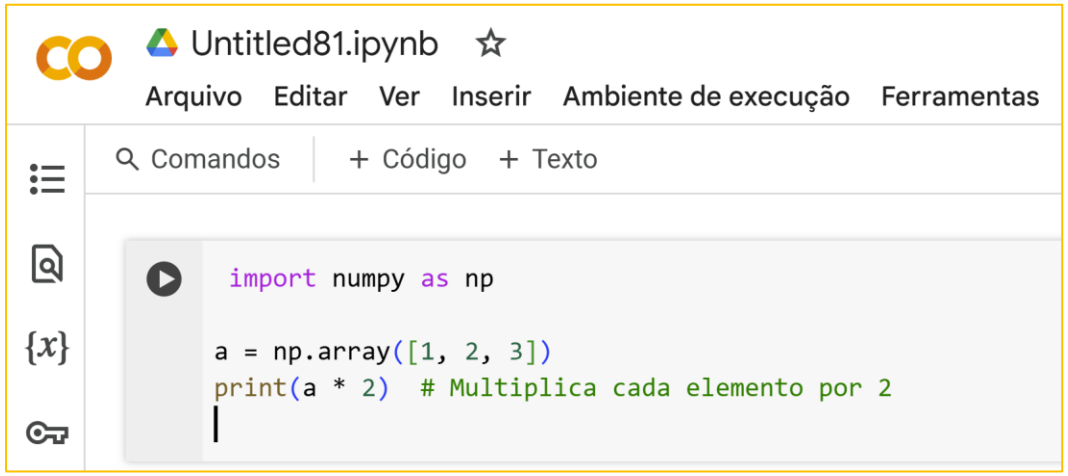
✅ **Compartilhamento fácil** → Semelhante ao Google Docs, permitindo colaboração em tempo real.

✅ **Bibliotecas pré-instaladas** → Vem com várias bibliotecas como NumPy, Pandas, TensorFlow e Matplotlib.

Google Colab (*Google Colaboratory*)

https://colab.research.google.com/





Google Colab (*Google Colaboratory*)

Exemplo: Inserir a Célula de Código no Colab para testar o programa:

```
import numpy as np

a = np.array([1, 2, 3])
print(a * 2)  # Multiplica cada elemento por 2
```

Atividade:

- Explorar o Colab;
- Uso de Gemini;

Desvio Condicional

Comando **if / else** :

```
# prompt: create an example to a program to
read an integer value given by the user and
present if it is different of 0 or not

# Get input from the user
number = int(input("Enter an integer: "))
```

input("Digite um inteiro: "): Esta função exibe a mensagem "Digite um inteiro: " para o usuário e espera que ele digite algo e pressione Enter. A entrada é recebida como uma string.

int(...): Esta função tenta converter a string recebida do usuário em um inteiro. Se a entrada não for um inteiro válido (como “ABCD"), o programa encontrará um erro.

number = ...: O valor inteiro resultante é armazenado em uma variável chamada number.

```
if number != 0:
    print("The number is different from 0.")
else:
    print("The number is 0.")
```

if number != 0:: Esta é uma declaração condicional.
if: Esta palavra-chave introduz a condição.
number != 0: Esta é a condição que está sendo verificada. Significa "se number não for igual a 0".
: Estes dois pontos marcam o início do bloco de código que será executado se a condição for True.
If the condition in the if statement is True (meaning the number is not 0), this line will execute, printing the message to the console.
else: Esta palavra-chave introduz o bloco de código alternativo que será executado se a condição if for False.

Desvio Condicional

Comando **if / elif /else** :

```
value1 = float(input("Enter the first value: "))
value2 = float(input("Enter the second value: "))

if value1 > value2:
  print(f"{value1} is bigger than {value2}")
elif value2 > value1:
  print(f"{value2} is bigger than {value1}")
else:
  print(f"{value1} and {value2} are equal")
```

input("Digite o primeiro valor: "): Esta linha exibe a mensagem "Digite o primeiro valor: " e espera que o usuário digite algo e pressione Enter.
float(...): A função float() converte a entrada do usuário (que é inicialmente tratada como texto) em um número de ponto flutuante, permitindo valores decimais.
value1 = ...: O resultado (o número do usuário como um float) é armazenado em uma variável chamada value1.

if value1 > value2:: Esta linha inicia uma instrução if. Se value1 for maior que value2, o código abaixo desta linha será executado.
print(f"{value1} is larger than {value2}"): Esta linha imprime uma mensagem indicando que value1 é maior que value2. A sintaxe f"" permite que você incorpore variáveis diretamente na string.
elif value2 > value1:: elif significa "else if". Se a primeira condição (value1 > value2) for False, esta condição será verificada. Se value2 for maior que value1, o código recuado abaixo desta linha será executado.
print(f"{value2} is larger than {value1}"): Semelhante a antes, esta linha imprime uma mensagem indicando que value2 é maior que value1.
else:: Se nenhuma das condições anteriores for True (significando que value1 e value2 são iguais), o código recuado sob else será executado.
print(f"{value1} e {value2} são iguais"): Esta linha imprime uma mensagem indicando que os dois valores são iguais.

Laços de Repetição (Loop)

Comandos **For** e **While** :

Os loops **for** são geralmente preferidos para iterar sobre coleções como arrays quando se conhece o número de iterações (repetições) com antecedência. Eles são mais concisos e menos propensos a erros, pois não precisa gerenciar manualmente o índice.

Os loops **while** são mais flexíveis e adequados quando o número de iterações não é conhecido de antemão, ou quando se precisa de mais controle sobre a execução do loop com base em certas condições.

Obs: usar um loop **for** com enumerate ou range(len(...)) seria considerado mais “Pythonic” e idiomático para iterar sobre um array.

Este trecho de código demonstra como acessar elementos individuais de uma matriz unidimensional (como my_array) usando um loop for e seus respectivos índices. Aqui está uma explicação passo a passo:

```
for i in range(len(my_array)):
  print(f"Element at index {i}:{my_array[i]}")
```

Esta sequência representa essencialmente os índices dos elementos na matriz.
for i in ... atribui cada número desta sequência à variável i durante cada iteração do loop. Então, i assumirá sucessivamente os valores dos índices dos elementos da matriz.

Exemplo:

Se my_array contivesse os valores [10, 20, 30, 40, 50], a saída deste loop seria:

Element at index 0: 10
Element at index 1: 20
Element at index 2: 30
Element at index 3: 40
Element at index 4: 50

Esta linha inicia um loop for que itera sobre uma sequência de números gerada por range(len(my_array)).
len(my_array) fornece o comprimento (número de elementos) da matriz my_array.
range(len(my_array)) cria uma sequência de números de 0 até (mas não incluindo) o comprimento da matriz.

print(f"Elemento no índice {i}: {my_array[i]}")

Dentro do loop, esta linha é executada repetidamente para cada valor de i.
print(...) é uma função usada para exibir a saída no console.
f"Elemento no índice {i}: {my_array[i]}" é uma f-string (literal de string formatada). É uma maneira conveniente de incorporar variáveis ou expressões em uma string.
{i} é substituído pelo valor atual do índice i.
{my_array[i]} é substituído pelo valor do elemento em my_array localizado no índice i.
Em essência, este loop faz o seguinte:

Ele começa com o primeiro elemento da matriz (índice 0).
Ele imprime uma mensagem mostrando o índice atual (i) e o valor do elemento naquele índice (my_array[i]).
Ele repete as etapas 1 e 2 para cada elemento subsequente na matriz, incrementando o índice i a cada iteração.

Este trecho de código demonstra outra maneira de acessar elementos individuais de uma matriz unidimensional (my_array) usando um loop for, junto com seus índices, usando a função enumerate. Essa abordagem é frequentemente considerada mais Pythonic e legível.

```
for i, value in enumerate(my_array):
  print(f"Element at index {i}: {value}")
```

Esta linha inicia um loop for que itera sobre a saída da função enumerate(my_array).enumerate(my_array) cria um objeto iterável que produz pares de valores: o índice de um elemento e o próprio elemento, para cada elemento em my_array.for i, value in ... atribui o índice à variável i e o valor do elemento à variável value para cada par gerado por enumerate.

Este loop faz o seguinte: Ele recupera o primeiro elemento do array junto com seu índice (0). Ele imprime uma mensagem exibindo o índice e o valor do elemento. Ele repete estas etapas para cada elemento subsequente no array, incrementando automaticamente o índice.

print(f"Element at index {i}: {value}")Esta linha é executada repetidamente dentro do loop para cada par índice-valor.print(...) exibe a saída no console.f"Elemento no índice {i}: {valor}" é uma string f usada para saída formatada.
{i} é substituído pelo valor do índice atual.
{valor} é substituído pelo valor do elemento atual.

Se my_array contivesse os valores [10, 20, 30, 40, 50], a saída deste loop seria:

Element at index 0: 10
Element at index 1: 20
Element at index 2: 30
Element at index 3: 40
Element at index 4: 50

Principal vantagem de usar enumerate:

Ele fornece diretamente o índice e o valor de cada elemento, tornando o código mais limpo e legível em comparação ao gerenciamento manual de índices com range(len(...)).

Inicializa-se um índice de variável para 0 para manter o controle do índice do elemento atual no array.
O loop while continua a ser executado enquanto a condição index < len(my_array) for True. Esta condição garante que o loop itere até que o índice atinja o comprimento do array (que é o índice do último elemento + 1).

```
import numpy as np

# Create a one-dimensional array with 10 random values
my_array = np.random.rand(10)

# Accessing elements using a while loop
index = 0
while index < len(my_array):
    value = my_array[index]
    print(f"Element at index {index}: {value}")
    index += 1   # Increment the index to move to the next element
```

Dentro do loop, o elemento no índice atual é acessado usando my_array[index] e o armazenamos na variável value.
Em seguida, apresenta o índice e o valor do elemento usando uma string f.

A linha index += 1 é necessária para incrementar o índice em 1 após cada iteração, garantindo que o loop passe para o próximo elemento no array. Sem esta linha, o loop se tornaria um loop infinito, acessando apenas o primeiro elemento repetidamente.

A saída será semelhante aos exemplos anteriores usando loops for:

Elemento no índice 0: [algum valor aleatório]
Elemento no índice 1: [algum valor aleatório]
...
Elemento no índice 9: [algum valor aleatório]

```python
# prompt: create an example of comand For to a bidimensional
array, sized 5x5 filled with sequencial values

import numpy as np

# Create a 5x5 array filled with sequential values
my_2d_array = np.arange(25).reshape(5, 5)

# Iterate through the 2D array using nested for loops
for i in range(my_2d_array.shape[0]):  # Iterate over rows
    for j in range(my_2d_array.shape[1]):  # Iterate over columns
        print(f"Element at row {i}, column {j}: {my_2d_array[i,
j]}")
```

Loop externo (i):

my_2d_array.shape[0] fornece o número de linhas na matriz (que é 5).
range(my_2d_array.shape[0]) cria uma sequência de números de 0 a 4 (representando os índices de linha).
O loop externo itera por cada índice de linha (i).

Loop interno (j):

my_2d_array.shape[1] fornece o número de colunas na matriz (que é 5).
range(my_2d_array.shape[1]) cria uma sequência de números de 0 a 4 (representando os índices de coluna).
Para cada linha (i), o loop interno itera por cada índice de coluna (j).

Criando uma função em Python

As funções em Python são definidas usando a palavra-chave **def**, seguida pelo nome da função, parênteses () e dois pontos:. O bloco de código dentro da função é recuado (identado).

Formato:

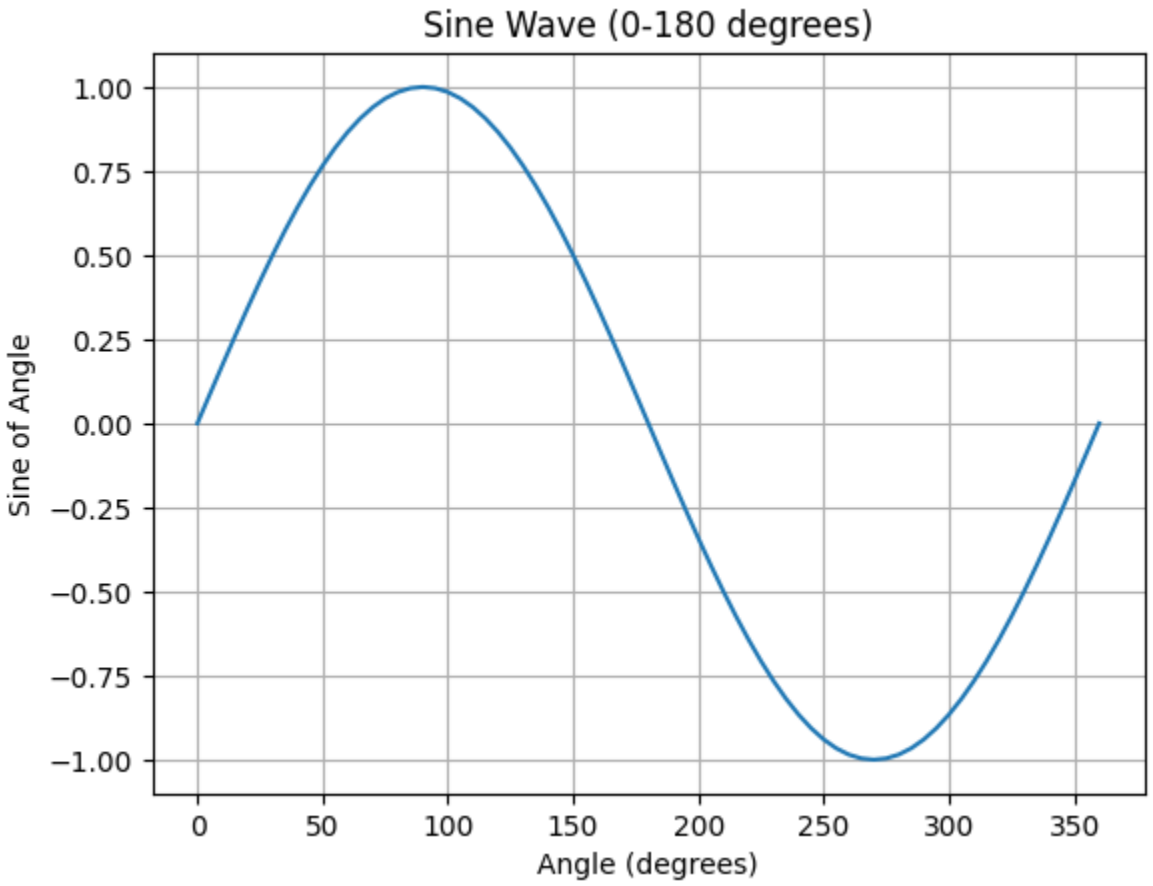
```
def function_name(parameters):
    """Docstring (optional)"""
    # Code to be executed
    return value  # Optional return statement
```

Exemplo:

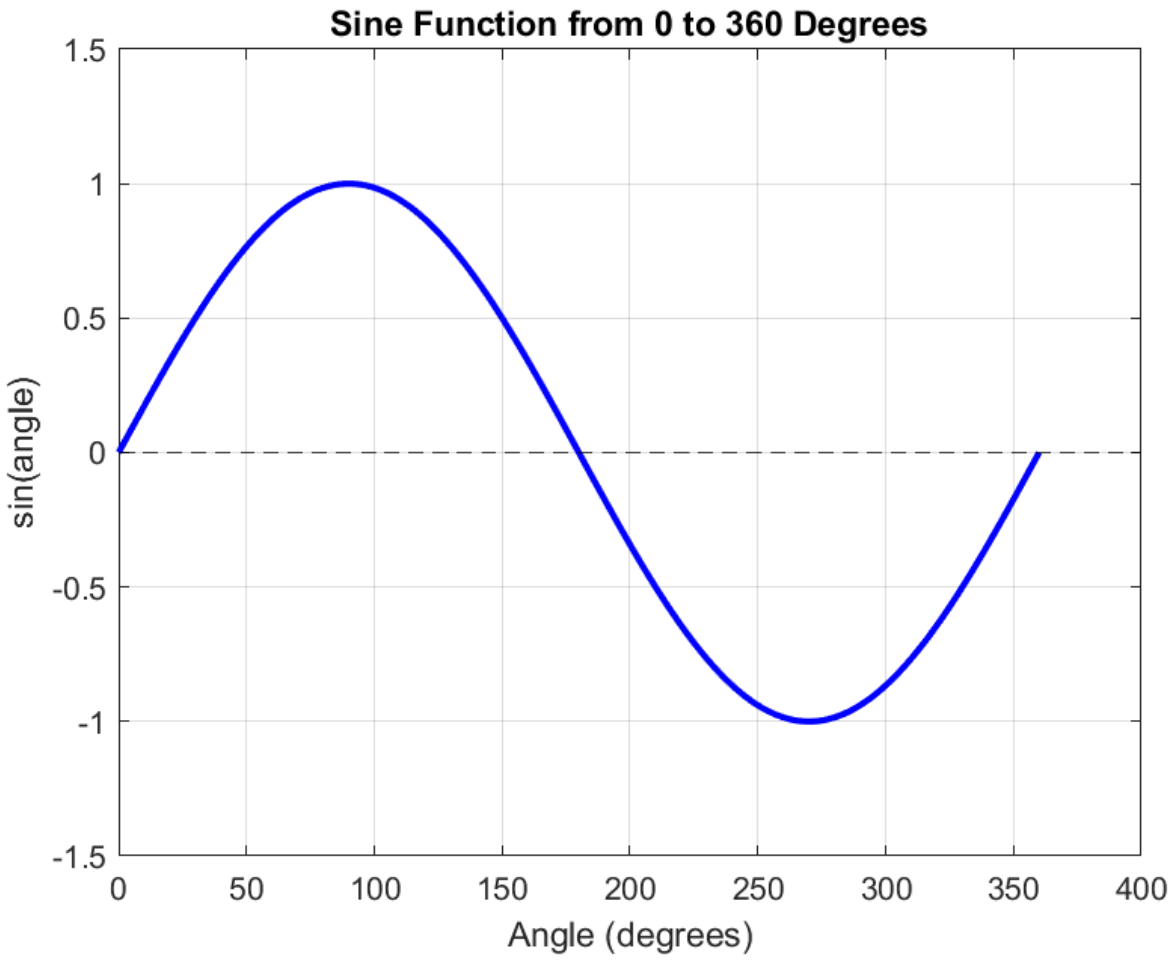
Criar uma matriz bidimensional com valores sequenciais.

```python
"""
Creates a bidimensional array with sequential values.

Args:
    rows: The number of rows in the array.
    cols: The number of columns in the array.

Returns:
    A NumPy array with the specified dimensions and sequential values.
"""


import numpy as np

def create_sequential_array(rows, cols):
    my_2d_array = np.arange(rows * cols).reshape(rows, cols)
    return my_2d_array

# Example usage
rows = 3
cols = 5
array = create_sequential_array(rows, cols)

# Print the array
for i in range(array.shape[0]):
    for j in range(array.shape[1]):
        print(f"Element at row {i}, column {j}: {array[i, j]}")
```

```python
# printing a graph to the sin of angle between 0 and 360

import matplotlib.pyplot as plt
import numpy as np

# Generate angles from 0 to 360 degrees
angles = np.arange(0, 361, 5)

# Calculate the sine of each angle
sin_values = np.sin(np.radians(angles))
print(sin_values)
# Create the plot
plt.plot(angles, sin_values)

# Add labels and title
plt.xlabel("Angle (degrees)")
plt.ylabel("Sine of Angle")
plt.title("Sine Wave (0-360 degrees)")

# Display the plot
plt.grid(True)
plt.show()
```



```
% Define the angle range from 0 to 360 degrees
angles = 0:360;

% Calculate the sine of each angle (converting degrees to radians)
sine_values = sin(deg2rad(angles));

% Create the plot
figure;
plot(angles, sine_values, 'b-', 'LineWidth', 2);
grid on;

% Add title and labels
title('Sine Function from 0 to 360 Degrees');
xlabel('Angle (degrees)');
ylabel('sin(angle)');

% Set the y-axis limits
ylim([-1.5 1.5]);

% Add a horizontal line at y = 0 for reference
yline(0, 'k--');

% Display the plot
hold off;
```



Exercícios

1. Exibição de Texto Simples

Crie um programa que exiba a seguinte mensagem usando print(): Bem-vindo ao curso de Python!

2. Exibição de Números e Operações Matemáticas

Escreva um programa que imprima o resultado da soma entre 15 e 30, mostrando a operação na saída.

Saída esperada:

15 + 30 = 45

3. Uso de Variáveis na Impressão

Crie um programa que armazene o nome e a idade de uma pessoa em variáveis e exiba a seguinte saída usando print():

nome = "Carlos" idade = 25

Saída esperada:

Carlos tem 25 anos.

4. Formatação de Números Decimais

Escreva um programa que exiba o valor de π com apenas **duas casas decimais**, usando print().

Dica: Utilize round() ou formatação de strings.

Saída esperada:

O valor de π é aproximadamente 3.14

5. Exibição com Quebra de Linha e Tabulação

Faça um programa que imprima o seguinte texto com \n para quebra de linha e \t para tabulação:

Saída esperada:

Lista de Compras: - Maçã - Pão - Leite

6. Impressão com f-strings

Escreva um programa que armazene o nome de um aluno e sua nota em variáveis e exiba uma mensagem formatada com **f-strings**.

aluno = "Ana" nota = 9.5

Saída esperada:

A aluna Ana obteve a nota 9.5 na prova.

6. Impressão com f-strings

Crie um programa em Python que armazene os **nomes de 5 alunos** e suas respectivas **quatro notas**. O programa deve calcular a **média das notas** de cada aluno e exibir os dados no formato de uma **tabela**, utilizando f-strings para alinhar as informações corretamente.

Exemplo de Entrada:

Alunos = ["Lucas", "Ana", "Pedro", "Mariana", "João"]
notas = [[8.5, 7.0, 9.2, 6.8], # Notas de Lucas
[6.0, 8.5, 7.4, 9.1], # Notas de Ana
[7.5, 9.0, 8.8, 6.5], # Notas de Pedro
[9.2, 8.3, 7.7, 8.9], # Notas de Mariana
[5.5, 6.8, 7.0, 6.2] # Notas de João]

Saída Esperada:

```
+------------+------+------+------+------+-------+
| Aluno      |  N1  |  N2  |  N3  |  N4  | Média |
+------------+------+------+------+------+-------+
| Lucas      |  8.5 |  7.0 |  9.2 |  6.8 |  7.88 |
| Ana        |  6.0 |  8.5 |  7.4 |  9.1 |  7.75 |
| Pedro      |  7.5 |  9.0 |  8.8 |  6.5 |  7.95 |
| Mariana    |  9.2 |  8.3 |  7.7 |  8.9 |  8.53 |
| João       |  5.5 |  6.8 |  7.0 |  6.2 |  6.38 |
+------------+------+------+------+------+-------+
```

7. Impressão com f-strings

Crie um programa em Python que apresente a sequinte sequencia de somatório com N termos. Por exemplo, para N=5:

```
1/5! + 2/4! + 3/3! + 4/2! + 5/1!
```

8. Impressão com f-strings

Crie um programa em Python que apresente a sequencia dos N termos do somatório do exercício 7 e o respectivo resultado do somatório.

```
S = 1/N! + 2/(N-1)! + 3/(N-3)! + … (N-1)/2! + N/1!
```

9. Uso de Lista e Função

Escreva um programa em Python que armazene **uma lista com 10 valores inteiros**. O programa deve conter uma **função** chamada verificar_presenca(), que recebe a lista e um número fornecido pelo usuário e verifica se esse número está presente na lista.

A função deve retornar **True** se o número estiver na lista e **False** caso contrário. O programa deve exibir uma mensagem informando o resultado da busca.

Requisitos:

✅ A lista deve conter **10 valores inteiros predefinidos**.
✅ O usuário deve digitar um número para ser buscado na lista.
✅ Criar e utilizar a função verificar_presenca(lista, valor).
✅ Exibir uma mensagem indicando se o número foi encontrado ou não.

Exemplo de Entrada:

valores = [10, 25, 30, 45, 50, 65, 70, 85, 90, 100]

Usuário digita:

Digite um número para buscar: 45

Saída Esperada:

O número 45 está na lista!

10. Gráficos e Funções

Escreva um programa em **Python** para calcular a área abaixo de uma curva definida por uma **função matemática** utilizando o **método dos trapézios**. O programa deve:

1. Solicitar ao usuário os **limites de integração** (início e fim do intervalo).
2. Definir uma função matemática **f(x)** (exemplo: uma função quadrática como $f(x) = x^2 + 2x + 1$).
3. Dividir o intervalo em **n subintervalos** iguais.
4. Aplicar a **Regra do Trapézio** para calcular a área aproximada sob a curva.
5. Exibir o resultado da área calculada.
6. Exibir o gráfico da função. Testar para diferentes funções.

Fórmula do Método dos Trapézios:
A integral aproximada de uma função f(x) no intervalo [a,b] pode ser dada por:

$$A \approx \frac{h}{2} \sum_{i=0}^{n} (f(x_i) + f(x_{i+1}))$$

Onde:

- $h = \frac{b-a}{n}$ é o tamanho de cada subintervalo.
- n é o número de subdivisões (quanto maior, mais precisa será a aproximação).
- x_i são os pontos no intervalo.

**** Alunos que já dominam o Python, realizar os exercícios usando a programação no MatLab;
**** Realizar o Upload da Atividade no AVA (para quem tem acesso ao Sistema) ou me encaminhar por email (quem não tem acesso ao Sistema). Email: marcelo.rudek@pucpr.br

Activities: (English version)

1. Text Display

Create a program that displays the following message using print(): “Welcome to the Python course!”

2. Presenting Numbers and Mathematical Operations

Write a program that prints the result of the sum between 15 and 30, displaying the operation in the output.

Expected Output:

15 + 30 = 45

3. Using Variables in Printing

Create a program that stores a person's name and age in variables and displays the following output using print():

name = "Carlos" age = 25

Expected Output :

Carlos is 25 years old.

4. Formatting Decimal Numbers

Write a program that displays the value of π with only two decimal places, using print().

tip: Use round() or string formatting.

Expected Output :

The value of π is approximately 3.14

5. Display with Line Breaks and Tabs

Write a program that prints the following text using \n for line breaks and \t for tabs. :

Expected Output :

Shopping List: - Apple - Bread - Milk

6. Printing with f-strings

Write a program that stores a student's name and grade in variables and displays a message formatted with f-strings.

student = "Ana" grade = 9.5

Expected Output :

Student Ana obtained a grade of 9.5 on the test.

6. Printing with f-strings

Write a Python program that stores the names of 5 students and their respective four grades. The program should calculate the average grade for each student and display the data in a table format, using f-strings to align the information correctly.

Input Example:

Students = ["Lucas", "Ana", "Pedro", "Mariana", "João"]
grades = [[8.5, 7.0, 9.2, 6.8], # Lucas's grades
[6.0, 8.5, 7.4, 9.1], # Ana's grades
[7.5, 9.0, 8.8, 6.5], # Pedro's grades
[9.2, 8.3, 7.7, 8.9], # Mariana's grades
[5.5, 6.8, 7.0, 6.2] # João's grades]

Expected Output :

```
+------------+------+------+------+------+-------+
| Aluno      |  N1  |  N2  |  N3  |  N4  | Média |
+------------+------+------+------+------+-------+
| Lucas      |  8.5 |  7.0 |  9.2 |  6.8 |  7.88 |
| Ana        |  6.0 |  8.5 |  7.4 |  9.1 |  7.75 |
| Pedro      |  7.5 |  9.0 |  8.8 |  6.5 |  7.95 |
| Mariana    |  9.2 |  8.3 |  7.7 |  8.9 |  8.53 |
| João       |  5.5 |  6.8 |  7.0 |  6.2 |  6.38 |
+------------+------+------+------+------+-------+
```

7. Printing with f-strings

Create a Python program that displays the following summation sequence with N terms. For example, for N=5:

```
1/5! + 2/4! + 3/3! + 4/2! + 5/1!
```

8. Printing with f-strings

Write a Python program that displays the sequence of N terms in the summation from exercise 7 and the corresponding summation result.

```
S = 1/N! + 2/(N-1)! + 3/(N-3)! + … (N-1)/2! + N/1!
```

9. Using Lists and Functions

Write a Python program that stores a list of 10 integer values. The program should contain a function called check_presence(), which receives the list and a number provided by the user and checks if that number is present in the list.

The function should return True if the number is in the list and False otherwise. The program should display a message informing the result of the search..

Requirements:

✅ The list must contain 10 predefined integer values.
✅ The user must enter a number to be searched for in the list.
✅ Create and use the function check_presence(list, value).
✅ Display a message indicating whether the number was found or not..

Input Example:

values = [10, 25, 30, 45, 50, 65, 70, 85, 90, 100]

User types:

Enter a number to search: 45

Expected Output:

Number 45 is on the list!

10. Graphs and Functions

Write a Python program to calculate the area under a curve defined by a mathematical function using the trapezoidal rule. The program should:

1. Ask the user for the integration limits (beginning and end of the interval).
2. Define a mathematical function f(x) (example: a quadratic function such as $f(x) = x^2 + 2x + 1$).
3. Divide the interval into n equal subintervals.
4. Apply the Trapezoidal Rule to calculate the approximate area under the curve.
5. Display the result of the calculated area.
6. Display the graph of the function. Test for different functions.

Trapezoid Method Formula :
The approximate integral of a function f(x) in the interval [a,b] can be given by :

$$A \approx \frac{h}{2} \sum_{i=0}^{n} (f(x_i) + f(x_{i+1}))$$

Onde:

- $h = \frac{b-a}{n}$ é o tamanho de cada subintervalo.
- n é o número de subdivisões (quanto maior, mais precisa será a aproximação).
- x_i são os pontos no intervalo.

**** Students who are already proficient in Python should complete the exercises using MatLab programming;
**** Upload the activity to the AVA (for those with access to the system) or send it to me by email (for those without access to the system). Email: marcelo.rudek@pucpr.br